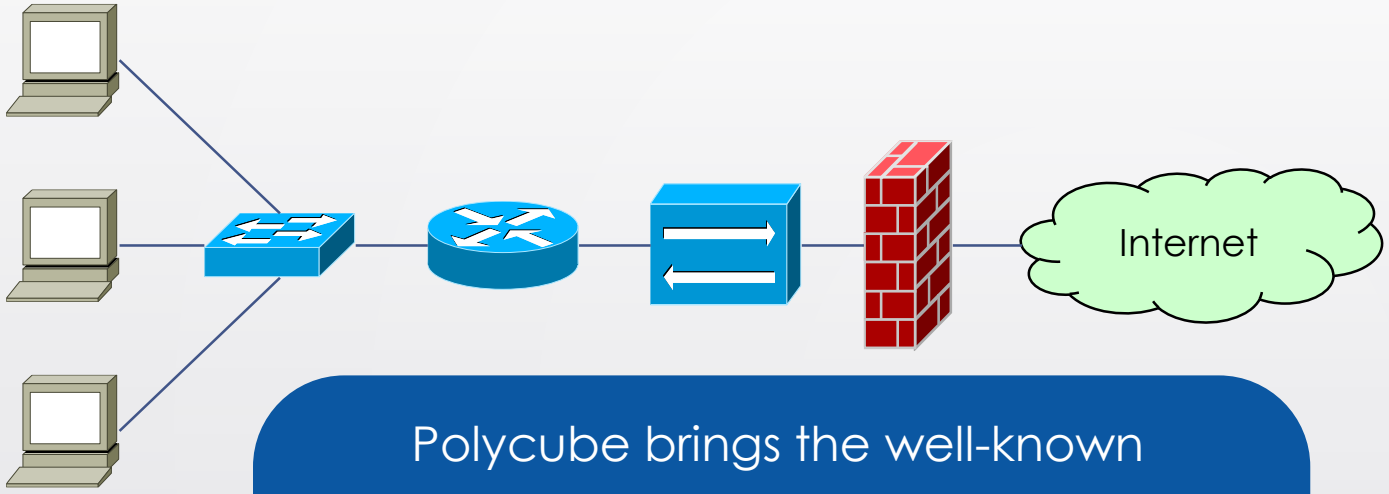
Have you Ever Tried to Create Complex Network Service Chains with eBPF?

F. Risso, Politecnico di Torino, Italy

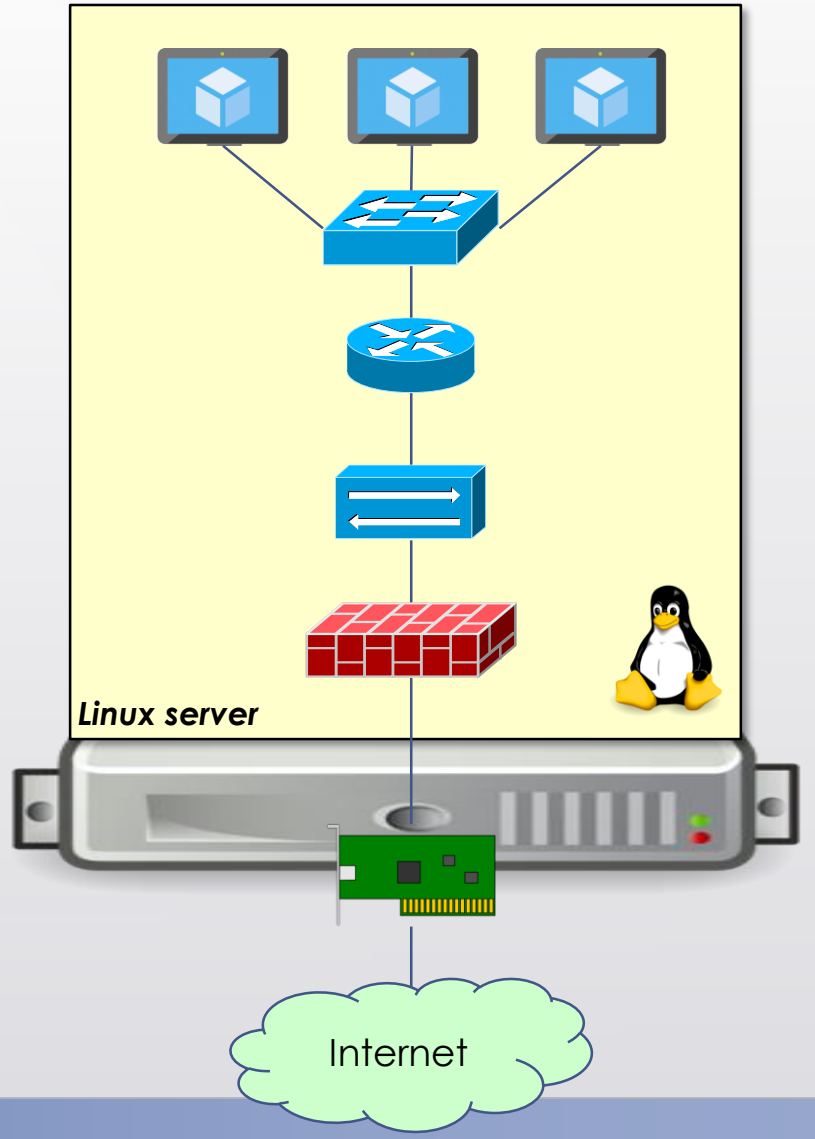
Yunsong Lu, Sebastiano Miano, Alex Palesandro, Jianwen Pi, Fulvio Risso, Aasif Shaikh, Mauricio Vasquez Bernal



From physical to virtual services



Polycube brings the well-known paradigms used in traditional networking, i.e., **boxes**, **ports**, **wires**, in the Linux virtualized world, without giving up on efficiency thanks to the **eBPF** in-kernel processing

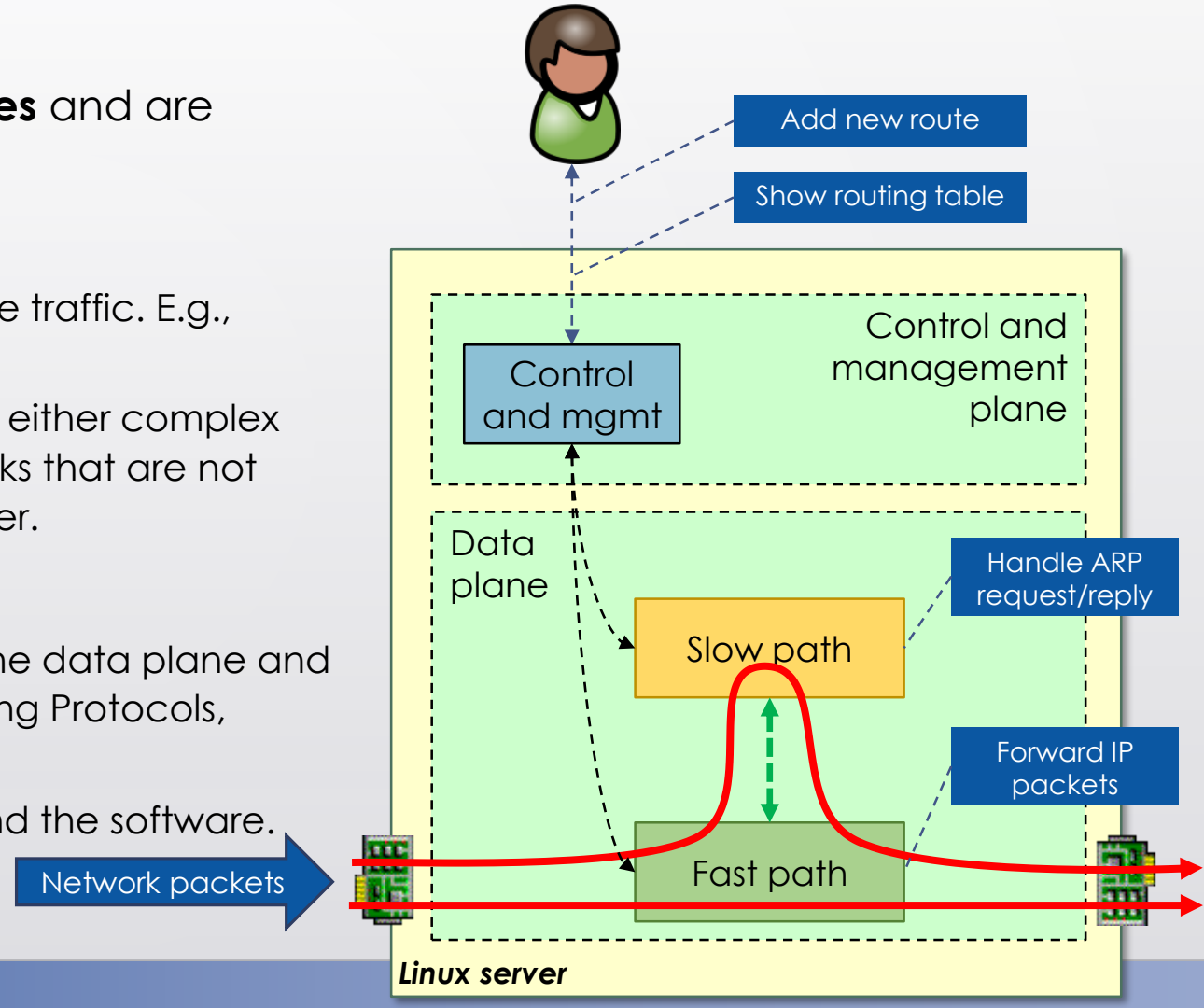


The path to Polycube

- Have you ever tried to create complex **network services** with eBPF?
  - A network service is not just data plane
- Have you ever tried to create **network topologies** with eBPF services?
  - People may need more than just a high-performance load balancer
- Can we simplify the life of **service developers**?
  - Abstractions, helpers (e.g., to hide differences between TC and XDP, etc)
- Can we simplify the life of **network admins**?
  - Single point of control, coherent network interfaces
- …without giving up on **performance**?
- … and leveraging the best of the **Linux ecosystem**?

Objective #1: Common structure and abstractions for NFs

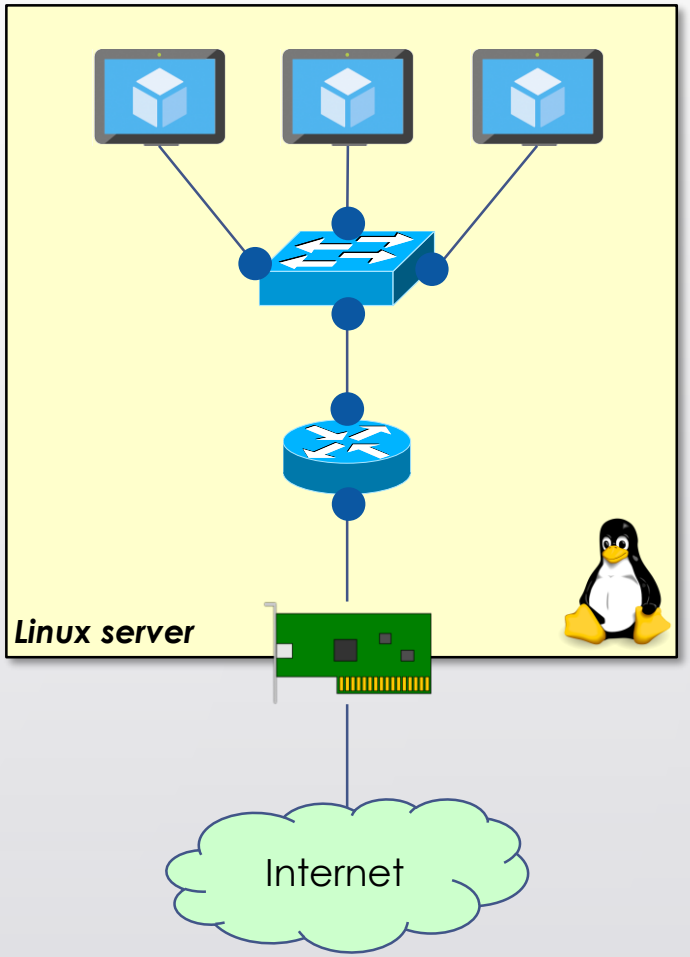
- Network functions in Polycube are called **cubes** and are structured in **three** components
- **Data** plane
  - **Fast path**: running in kernel, handling most of the traffic. E.g., packet forwarding in a router.
  - **Slow path**: running in user space, implementing either complex data plane tasks (slim fast path principle) or tasks that are not supported by eBPF. E.g., ARP handling in a router.
- **Control** and **management** plane
  - Control: out-of-band tasks needed to control the data plane and to react to possible complex events. E.g., Routing Protocols, Spanning Tree
  - Management: interaction between humans and the software. E.g., service configuration, statistics



Objective #2: Service Function Chaining

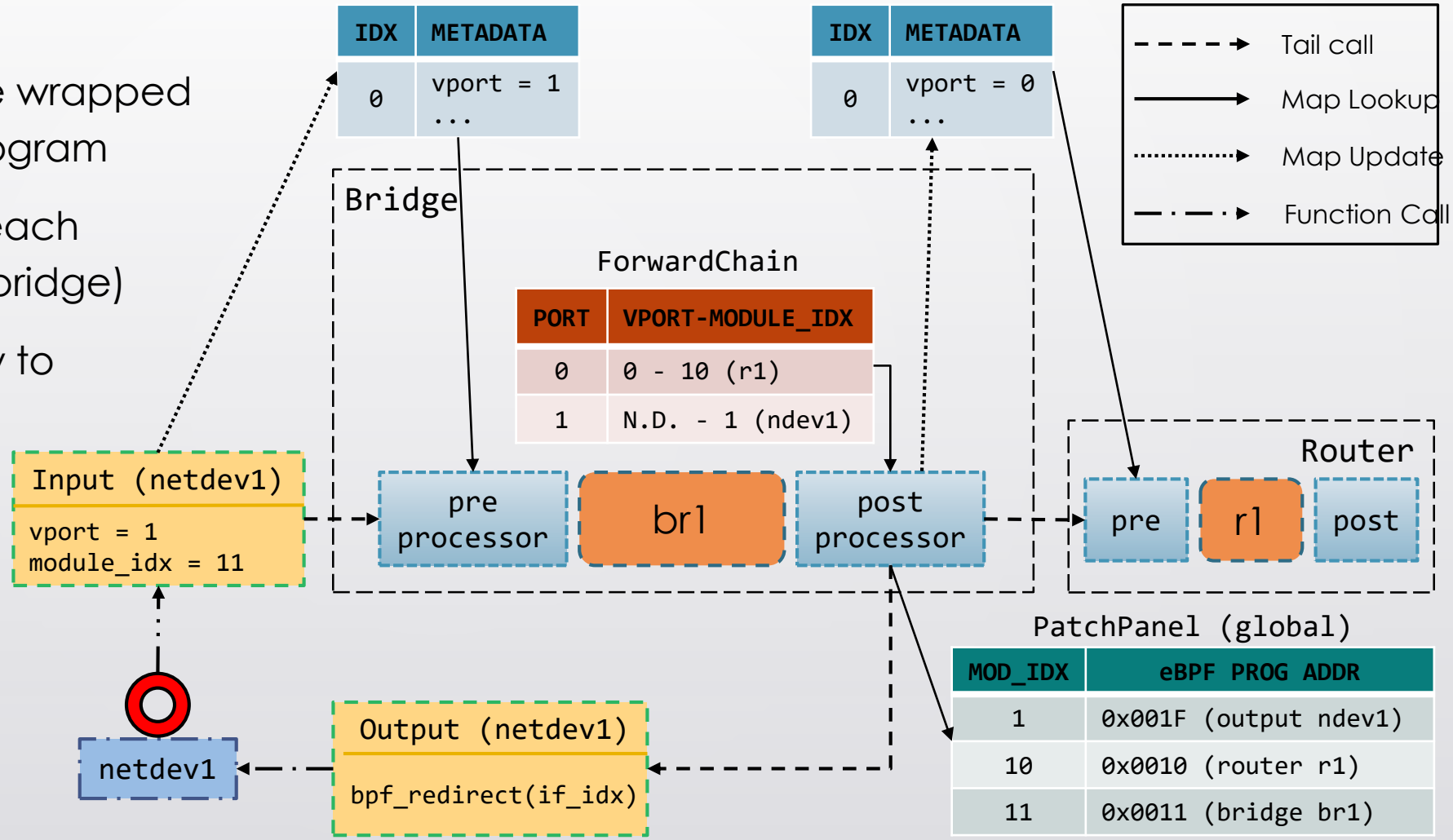
```
polycubectl simplebridge add br1
polycubectl br1 ports add to_veth1
polycubectl connect br1:to_veth1 veth1
polycubectl br1 ports add to_veth2
polycubectl connect br1:to_veth2 veth2
polycubectl br1 ports add to_veth3
polycubectl connect br1:to_veth3 veth3
polycubectl br1 ports add to_router

polycubectl router add r1
polycubectl r1 ports add to_br1
polycubectl connect r1:to_br1 br1:to_router
polycubectl r1 ports add to_internet
polycubectl connect r1:to_internet eth0
```



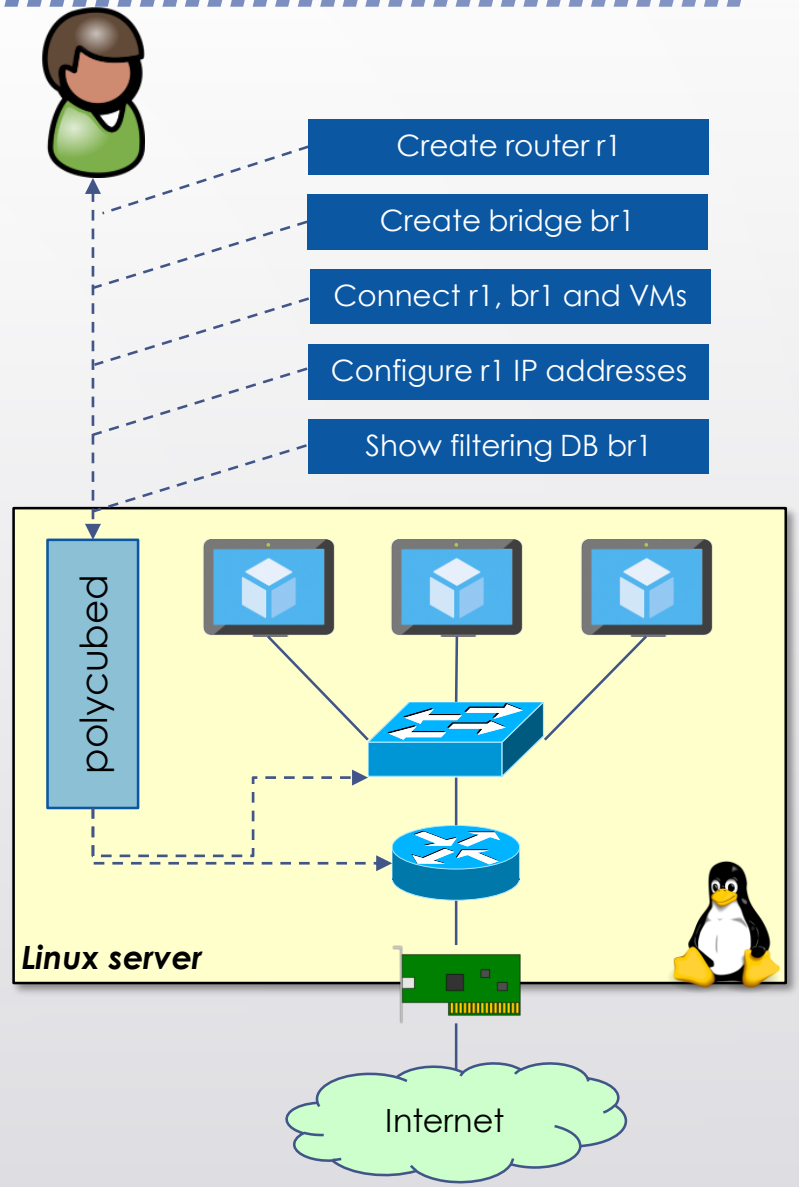
Objective #2: Service Function Chaining

- Physical/virtual ports are wrapped by an (hidden) eBPF program
- The same happens for each network function (e.g., bridge)
- Simplified CLI/REST allow to create service chains
- Simplified data plane helper functions allow to receive packets and send packet out (or redirect to slow path)



Objective #3: Single point of control

- **Single point of control** to handle the lifecycle of the entire **virtual network**
- **Topology**-related tasks
  - **Creating** services
  - **Connecting** services together and to physical/virtual interfaces
- **Service**-related tasks
  - **Configuring** each service
  - **Inspecting** run-time information and statistics of each service



Objective #4: **Model-Driven** Service Abstraction

- Simplify the **development** and the **execution** of new network services

Network function YANG data model

It defines the structure, syntax and semantics of the service data.

Automatic code generation

Command line interface (CLI)

Generic CLI, which can control any network function. CLI commands are automatically transformed in the corresponding REST API and sent to the controlling daemon.
The developer of the service does not have to write the CLI for its service.



REST API

{ REST }

REST API is automatically generated from the data model. Anyone can interact with the service (through REST) by just knowing the data model.
Incoming data can be automatically validated against the data model.

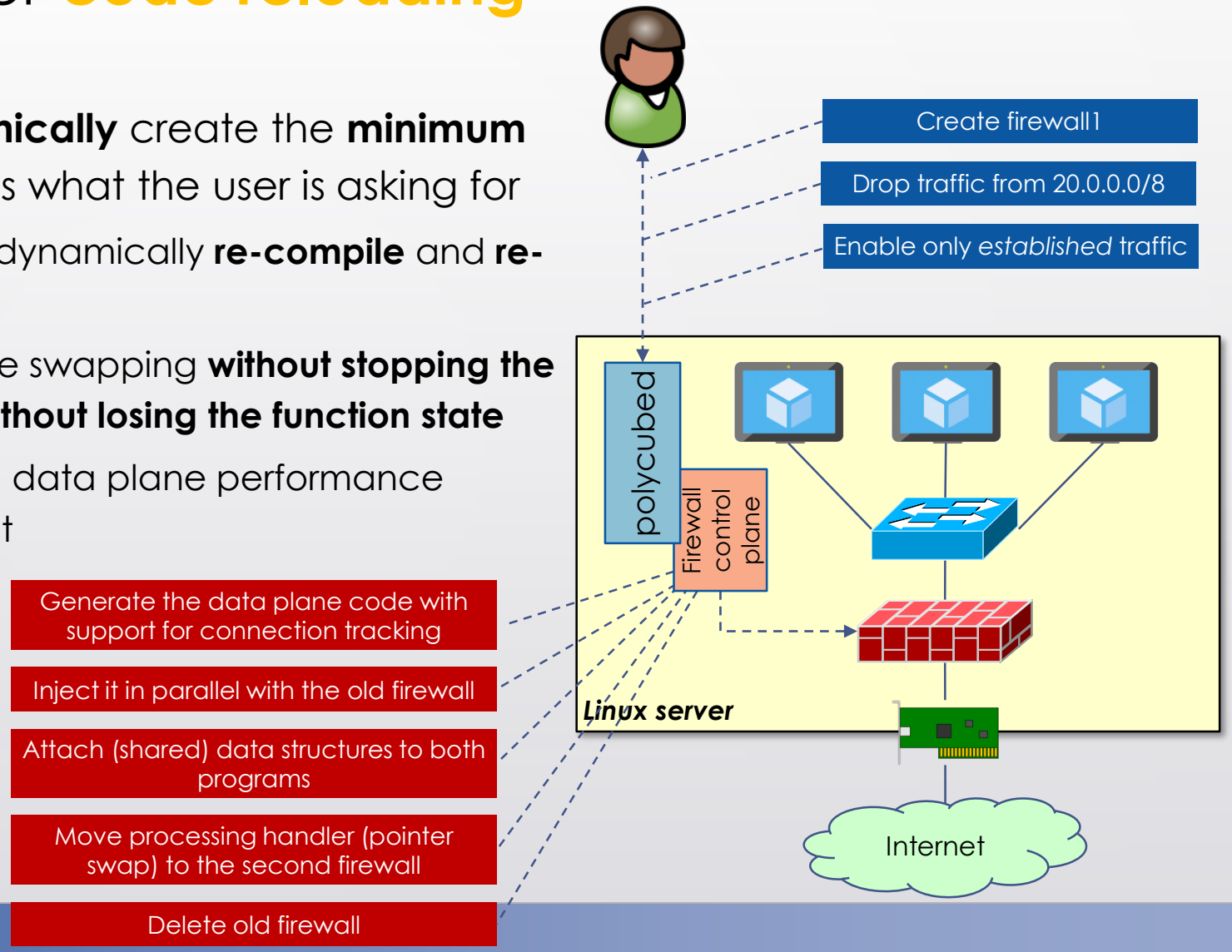


Service-specific source code

Several classes for each network function are automatically generated from the data model: objects, setter/getter, validators.
Developers can write the service logic (data model; fast, slow, ctrl path), while glue logic is provided by the framework.

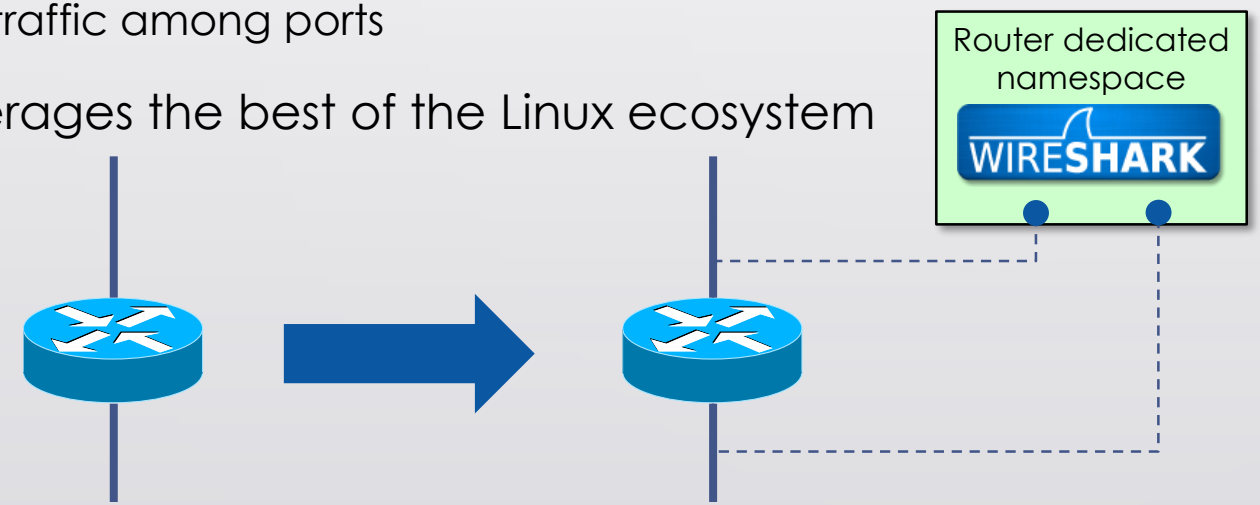
Objective #5: Support for code reloading

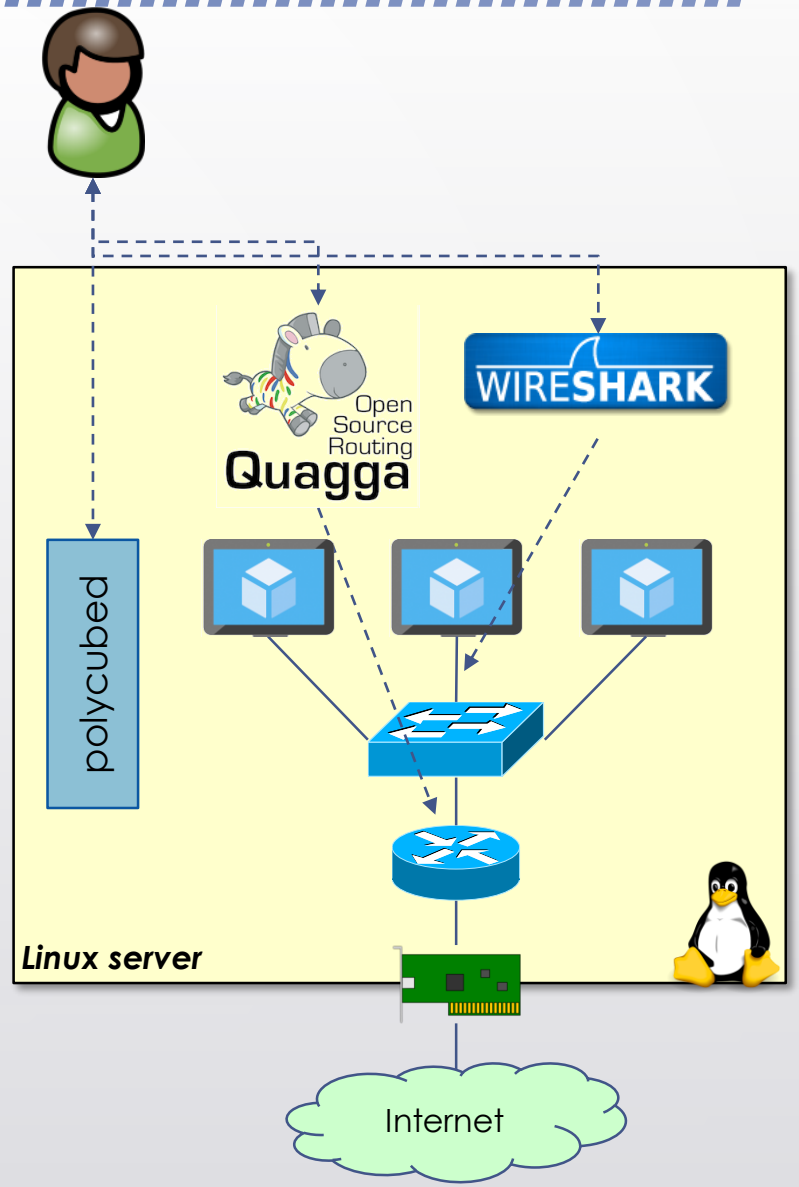
- Support for the capability to **dynamically** create the **minimum** data path code needed to process what the user is asking for
  - Enabled by the eBPF capability to dynamically **re-compile** and **re-inject** the data plane at run-time
  - Polycube enables data plane code swapping **without stopping the service** and, whenever possible, **without losing the function state**
  - Polycube can achieve the optimal data plane performance based on the requested feature set



Objective #6: Leverage Linux ecosystem

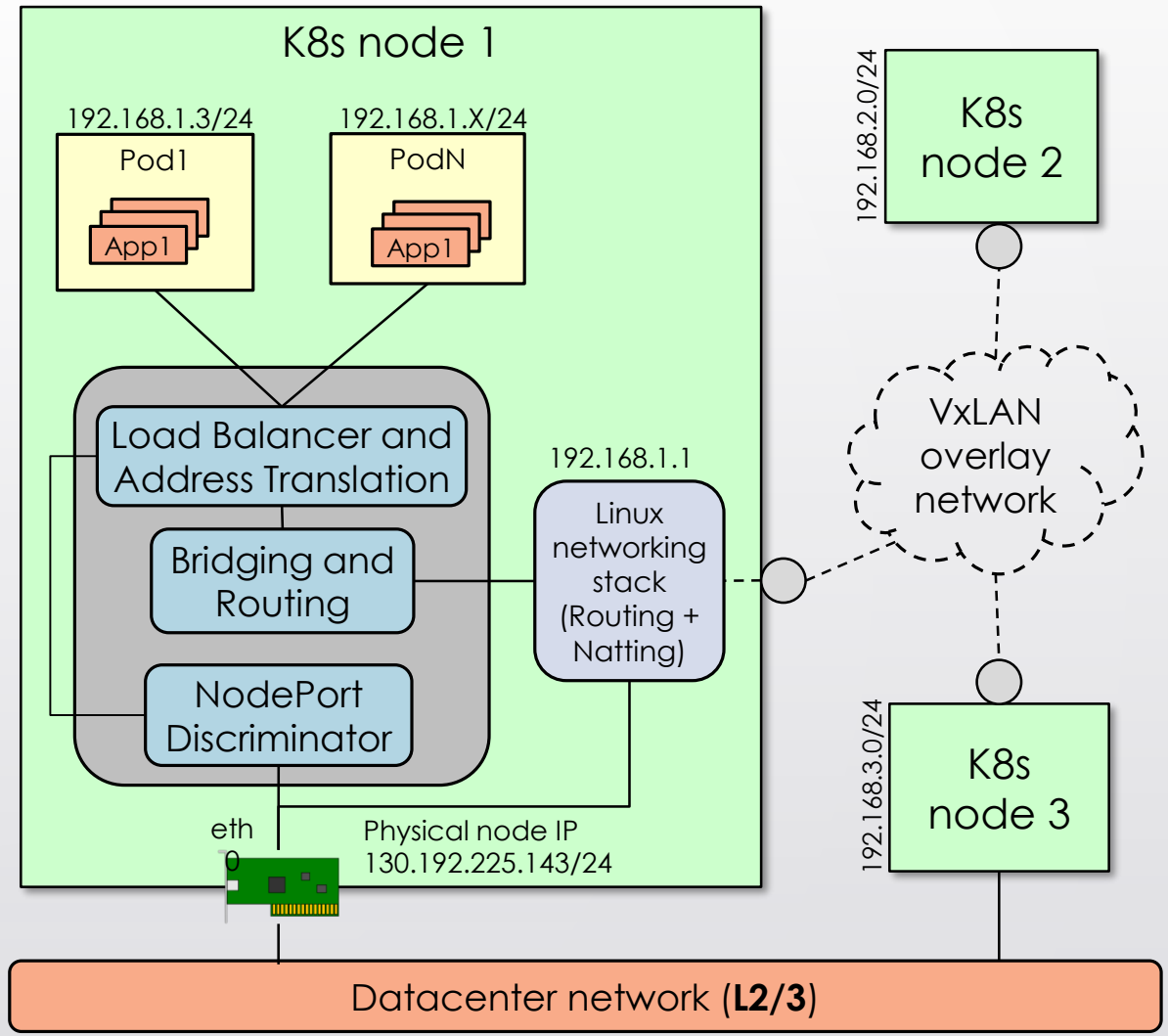
- A Polycube service running in "shadow" mode **duplicates** each port
  - The first ("real") port handles the traffic as usual
  - The second ("shadow") port terminates in a dedicated namespace, where your tools (e.g., Quagga) can be executed
  - A clever traffic mirroring algorithm duplicates only required traffic among ports
- Leverages the best of the Linux ecosystem



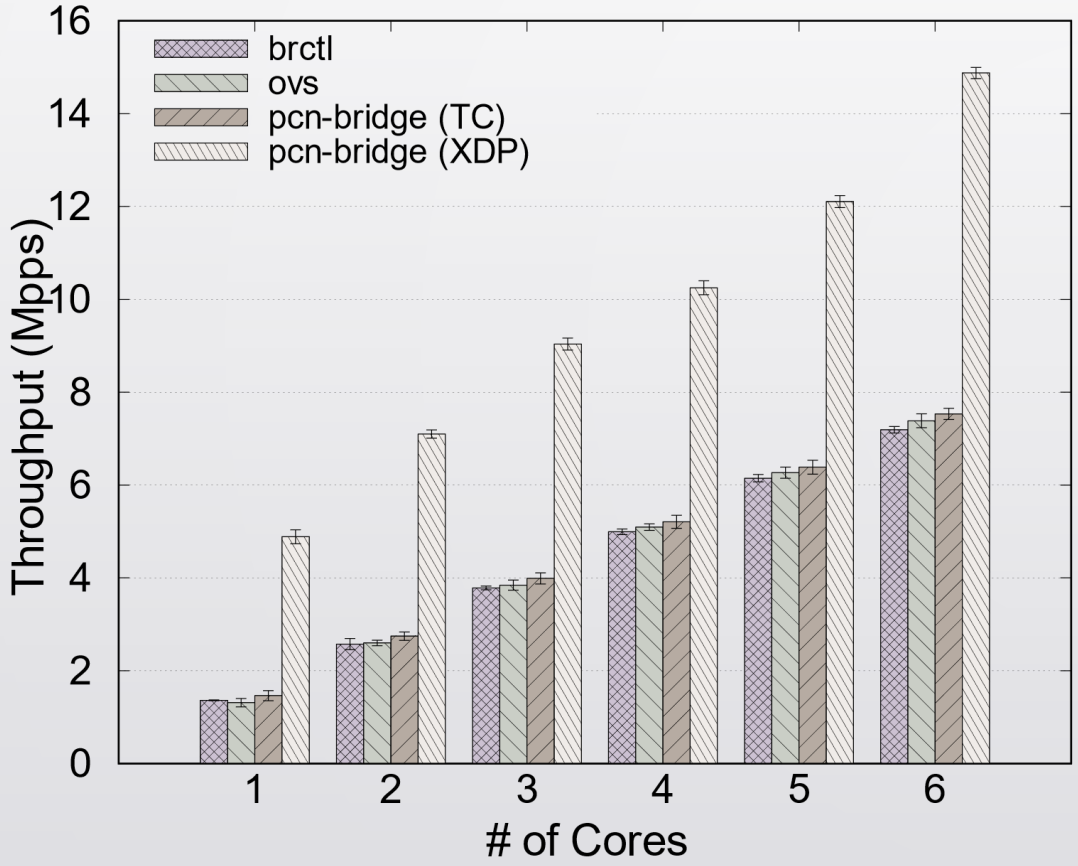


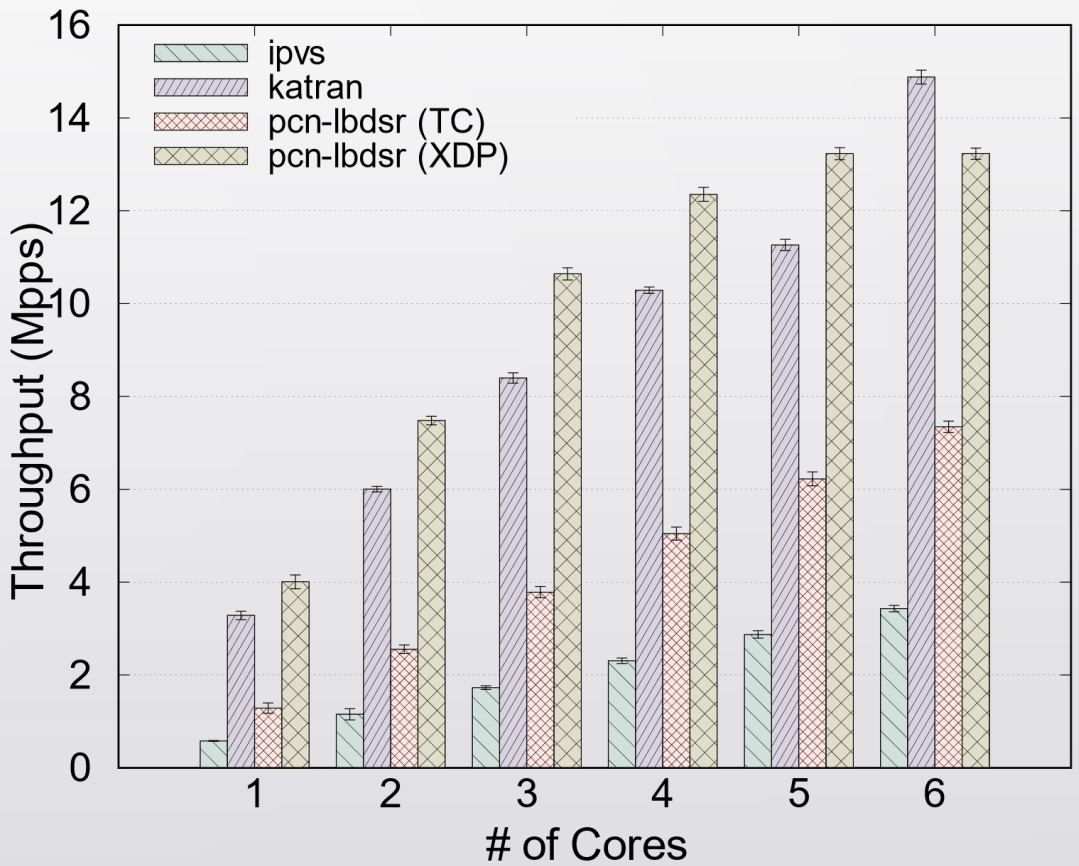
K8s Network Plugin Use Case

- Demonstrate the capability of creating complex network applications with Polycube
- Several independent Polycube services chained together
  - Some custom services to implement k8s specific functionality
- Full interaction with the Linux networking stack e.g., for tunneling and security purposes

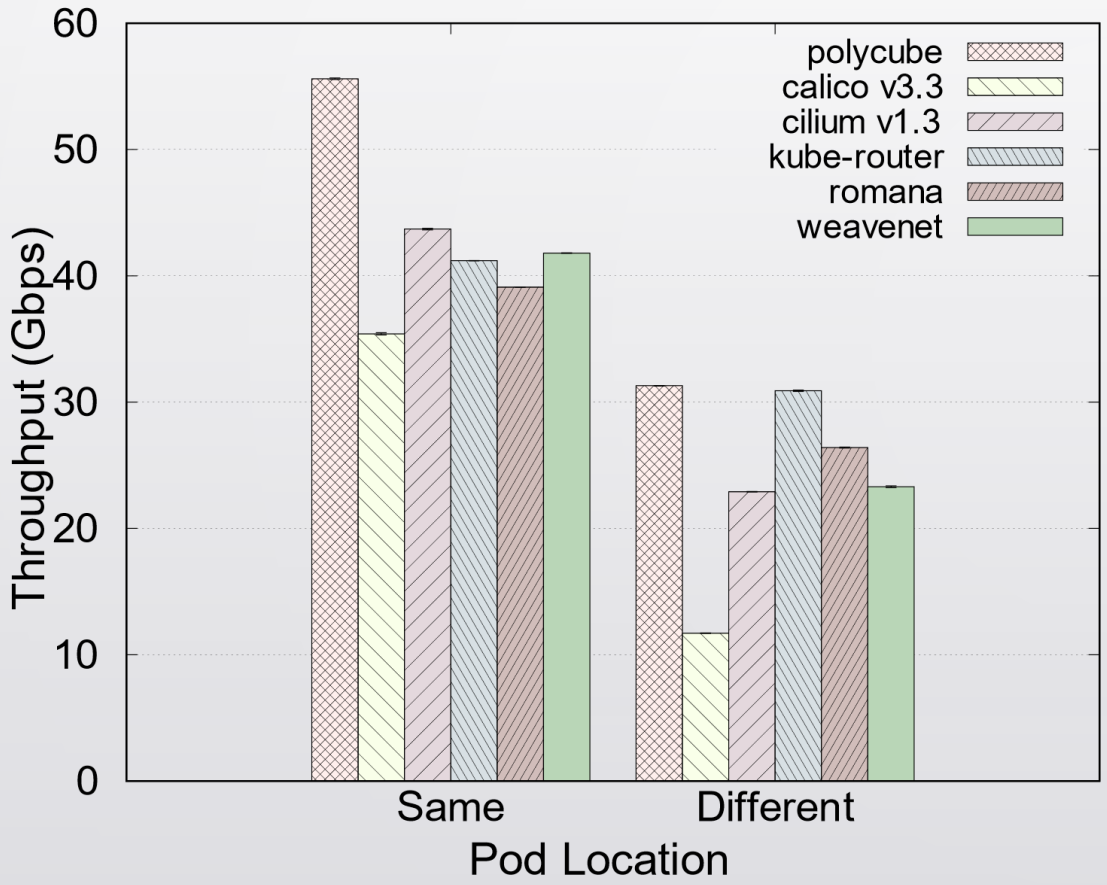


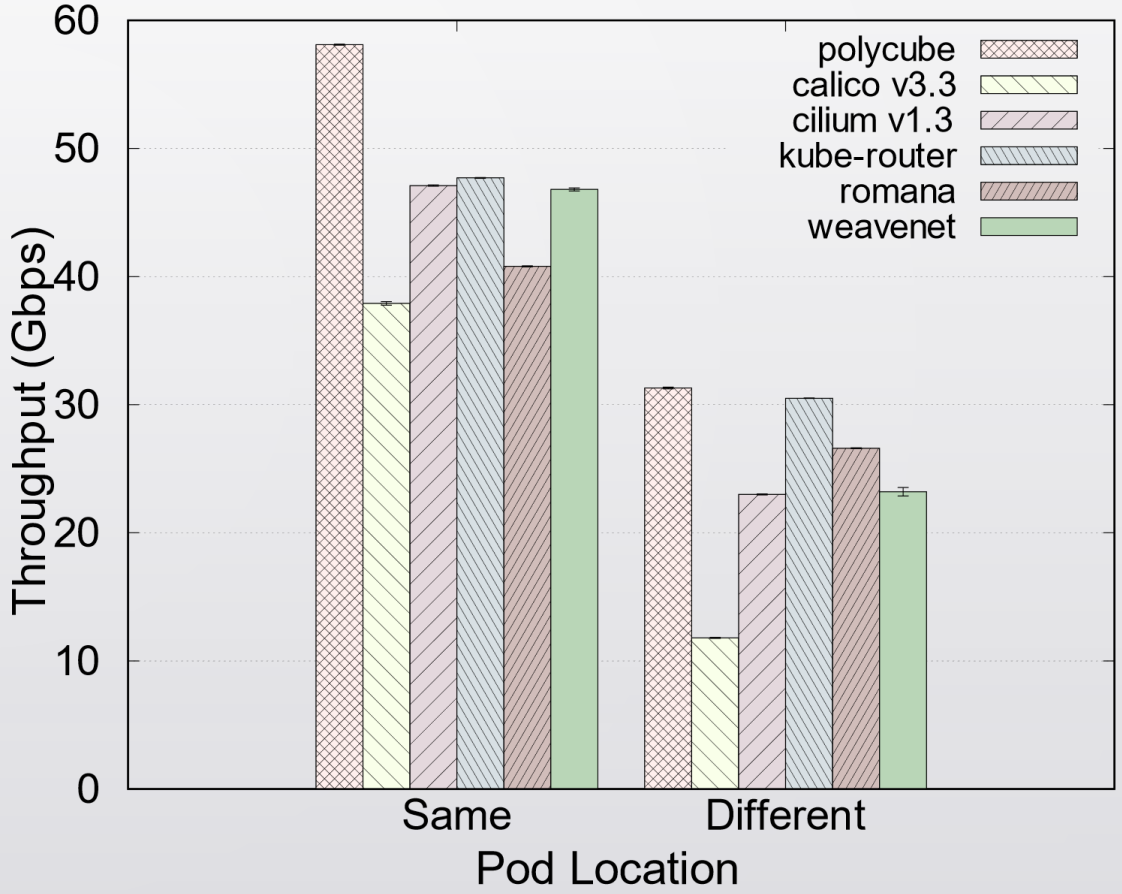
Evaluation of (some) Polycube Network Functions





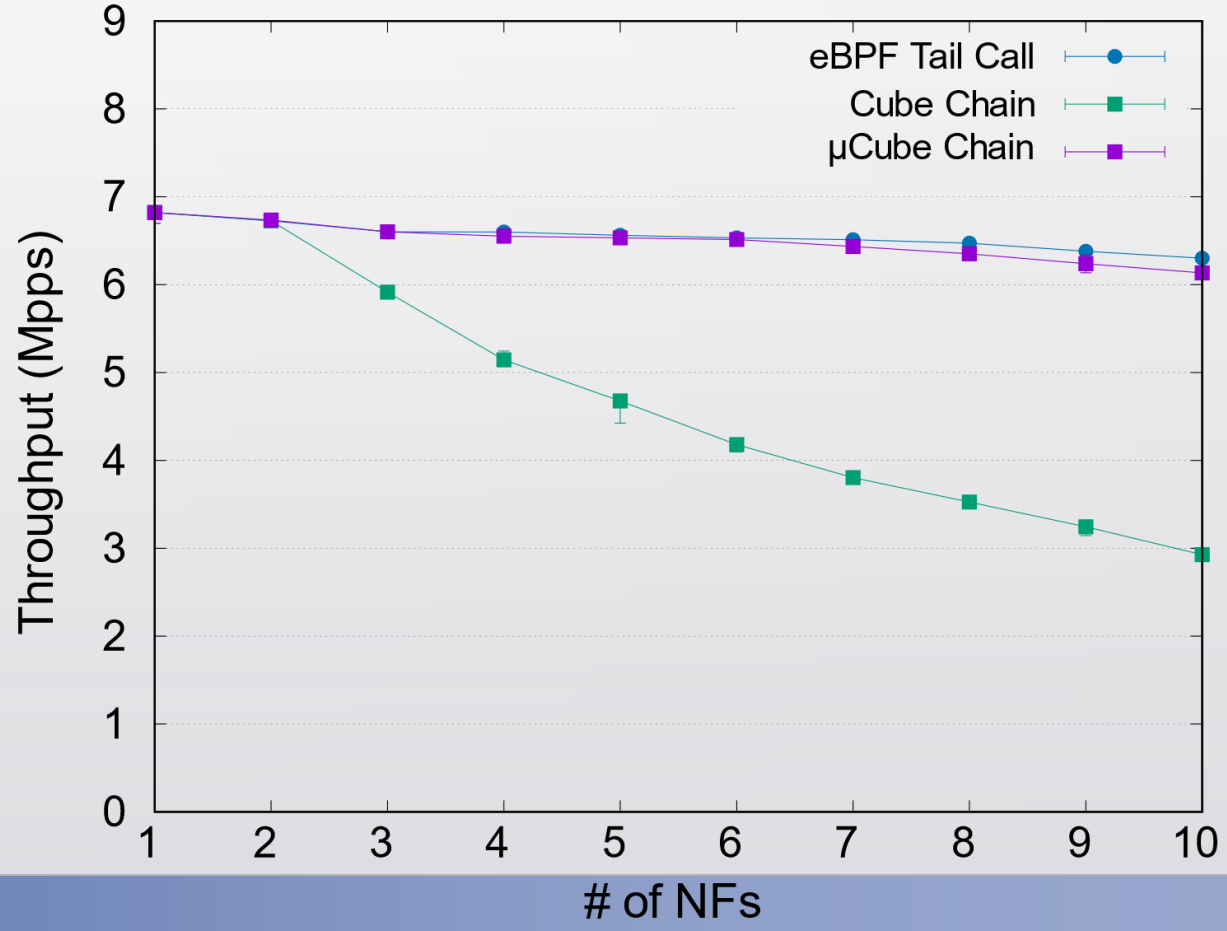
Polycube: K8s Plugin **Performance**





Polycube: Framework Overheads

Application	Through.	LoC (FP)	LoC (S/CP)
xdp_redirect	6.97Mpps	64	176
pcn_simplefwd (XDP)	6.86Mpps	53	56
tc_redirect	1.60Mpps	17	0
pcn_simplefwd (TC)	1.55Mpps	17	0



Project status

- Several PoC network functions
  - Policy-based forwarder
  - Simple bridge
  - Bridge
  - Router
  - NAT
  - DDoS mitigator (data plane only)
  - Firewall
  - Load balancer (RP)
  - Load balancer (DSR)
  - Packet capture service
  - Dynamic network monitor

- Actively developed at Politecnico di Torino
  - https://github.com/polycube-network/polycube/blob/master/AUTHORS
- Website: http://polycube.network
- Github: https://github.com/polycube-network/polycube

Polycube: Concluding Remarks

- First in-kernel NF framework based on eBPF
- Enables the creation of complex network services and network topologies
- Extend eBPF programming model with NF-specific abstractions
- Cubes can be dynamically injected/updated and optimized
- Leverages the power of the Linux ecosystem
- Performance
  - Low overhead compared to “vanilla” eBPF
  - Better performance compared to in-kernel alternatives
- Possibility to create complex in-kernel topologies by combining different services together
  - K8s Network Plugin